

การพัฒนากระบวนการอัตโนมัติสำหรับ การปฏิบัติการและประกันคุณภาพ (Auto Ops)

สาขาวิชาวิทยาศาสตร์และนวัตกรรมข้อมูล
วิทยาลัยสหวิทยาการ มหาวิทยาลัยธรรมศาสตร์





เอกสารประกอบการอบรม



Google Slide



Scan the QR code to join our LINE group



Workshop Agentic Living Lab 2026

วิทยากรบรรยาย



- ชื่อ : นางสาวสุภัทสรณ์ สุขภูติ
- หน่วยงาน: บริษัท เคซี ออล จำกัด
- ประวัติ:
 - ปริญญาตรี คณะวิทยาศาสตร์และเทคโนโลยี สาขาวิทยาการคอมพิวเตอร์
 - ตำแหน่ง Full-Stack Developer
- ช่องทางการติดต่อ:
 - praewsp1334@gmail.com

โครงการอบรมสำหรับนักศึกษาหลักสูตรวิทยาศาสตร์บัณฑิต

กำหนดการอบรม

- วันที่: วันที่ 8 มีนาคม 2569
เวลา: 08:30 - 16:00 น.
- สถานที่: ห้องปฏิบัติการและทาง Online

เวลา	กิจกรรม
วันที่ 8 มีนาคม 2569	การพัฒนาระบบอัตโนมัติสำหรับการปฏิบัติการและประกันคุณภาพ (Auto Ops)
08:30 - 09:00	ลงทะเบียนและรับเอกสารประกอบการอบรม
09:00 - 10:30	อบรมทฤษฎีแนวคิด CI/CD และ Container Orchestration
10:30 - 10:45	พักเบรก
10:45 - 12:00	ปฏิบัติการช่วงที่ 1: การสร้าง Docker Container และ K8s Pods
12:00 - 13:00	พักรับประทานอาหารกลางวัน
13:00 - 14:30	ปฏิบัติการช่วงที่ 2: การตั้งค่า CI/CD Pipeline บน GitLab/GitHub
14:30 - 14:45	พักเบรก
14:45 - 16:00	สรุปและถาม-ตอบก่อนปิดการอบรม

หลักการและเหตุผล

กระบวนการพัฒนาซอฟต์แวร์ในปัจจุบันมีความซับซ้อนและต้องอาศัยความรวดเร็ว การเตรียมสภาพแวดล้อมสำหรับการพัฒนา (Environment Setup) ด้วยวิธีการแบบเดิมที่ทำด้วยมือ (Manual) ทำให้เกิดความล่าช้าและความผิดพลาด อีกทั้งการขาดระบบตรวจสอบคุณภาพอัตโนมัติส่งผลให้เกิดปัญหาเมื่อนำระบบไปใช้งานจริง

เพื่อให้การดำเนินงานเป็นไปอย่างมีประสิทธิภาพ จึงจำเป็นต้องนำเทคโนโลยี DevOps และระบบอัตโนมัติ (Automation) มาประยุกต์ใช้ เพื่อลดระยะเวลาในการเตรียมการและยกระดับมาตรฐานการตรวจสอบคุณภาพซอฟต์แวร์ให้เป็นไปตามมาตรฐานสากล โครงการนี้มุ่งเน้นการสร้างกระบวนการทำงานแบบอัตโนมัติที่ช่วยลดภาระงานซ้ำซ้อน และเพิ่มความเชื่อมั่นในคุณภาพของระบบที่จะพัฒนาขึ้น

วัตถุประสงค์โครงการ

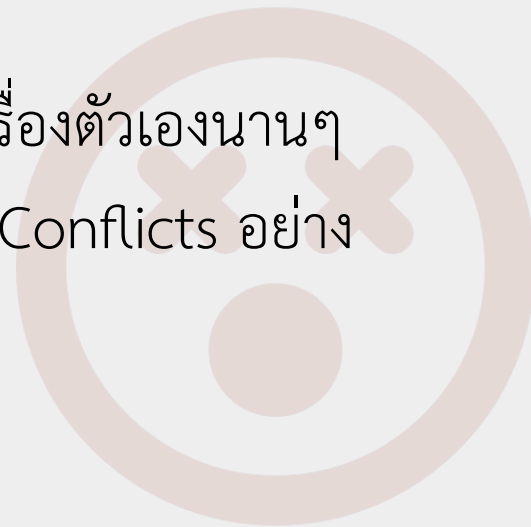
1. เพื่อพัฒนาระบบเตรียมสภาพแวดล้อมอัตโนมัติ (Auto-provisioning) สำหรับการพัฒนาซอฟต์แวร์
2. เพื่อติดตั้งระบบตรวจสอบคุณภาพโค้ดและสถานะระบบแบบเรียลไทม์ (QA Dashboard)
3. เพื่อถ่ายทอดความรู้ด้าน DevOps และ Container Technology แก่ผู้เข้าร่วม

แนวคิด CI/CD และ Container Orchestration

ปัญหาในการพัฒนาซอฟต์แวร์แบบดั้งเดิม (Traditional Development)

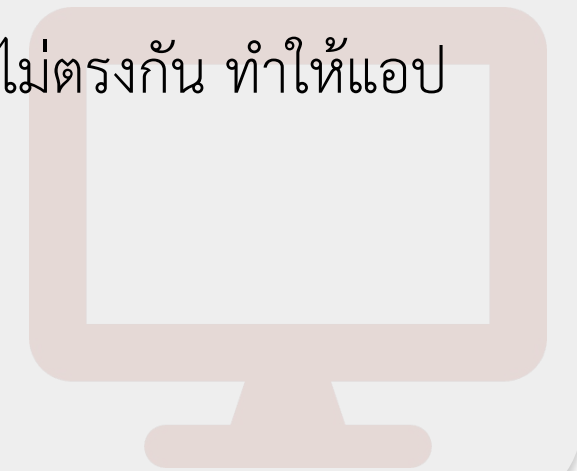
Integration Hell

การที่ Developer ต่างคนต่างเขียนโค้ดเก็บไว้ที่เครื่องตัวเองนานๆ (Long-lived branches) เมื่อนำมารวมกันจะเกิด Conflicts อย่างมหาศาล



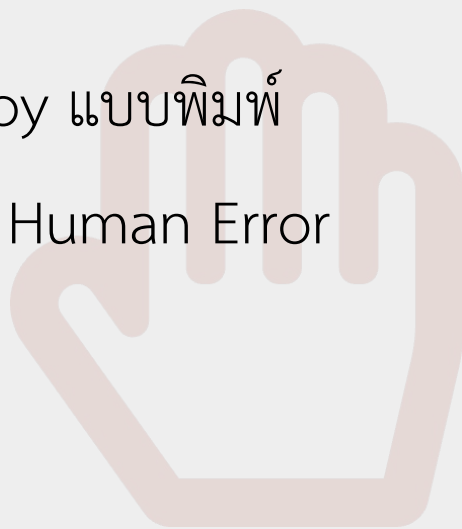
Environment Mismatch

Config หรือ Library ในเครื่อง Dev กับ Server ไม่ตรงกัน ทำให้แอปพังเมื่อขึ้น Production



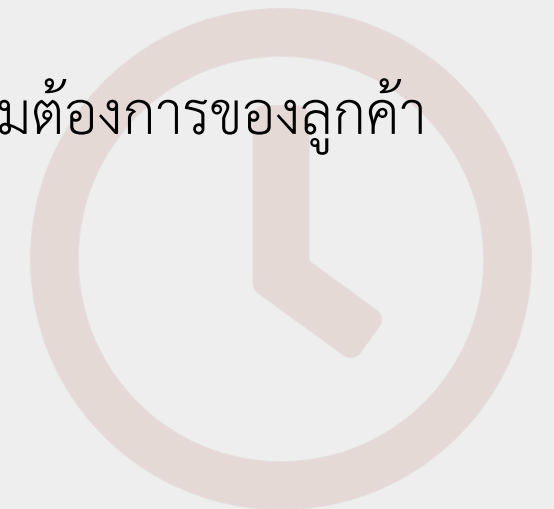
Manual Testing & Deploy

การทดสอบด้วยคน (Manual Test) และการ Deploy แบบพิมพ์คำสั่งเองที่ละบรรทัด มีความเสี่ยงสูง ล่าช้า และเกิด Human Error ได้ง่าย



Lack of Agility

รอบการปล่อยซอฟต์แวร์ล่าช้า ทำให้ไม่ทันต่อความต้องการของลูกค้าและตลาด



CI (Continuous Integration) คืออะไร

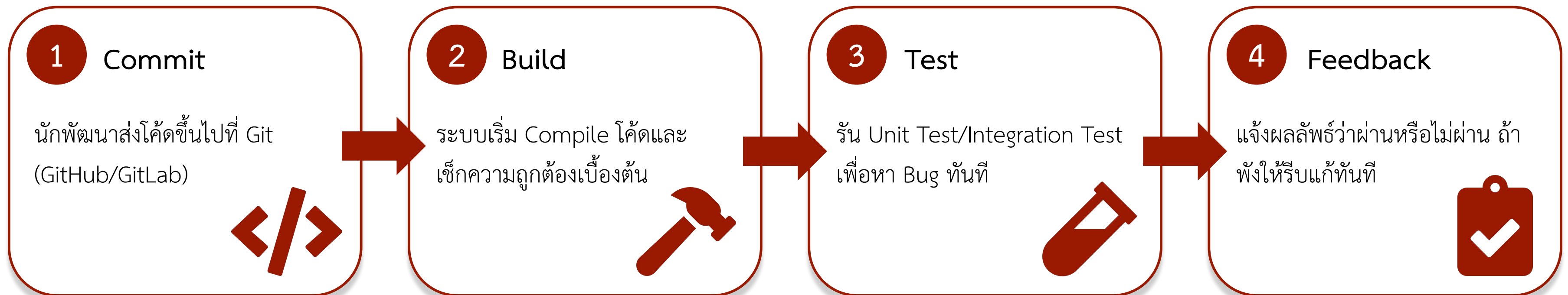
แนวปฏิบัติที่ให้นักพัฒนารวมโค้ดเข้าสู่ส่วนกลางบ่อยครั้งที่สุด พร้อมระบบตรวจสอบอัตโนมัติเพื่อให้แน่ใจว่าซอฟต์แวร์ยังคงทำงานได้ปกติทุกครั้งที่มีการแก้ไข

- ✓ ตรวจสอบข้อผิดพลาดได้เร็ว (Fall Fast)
- ✓ ลดความซับซ้อนในการรวมโค้ด

storemesh / poc / kanban-board / Pipelines				
Status	Pipeline	Created by	Stages	Actions
Failed 00:01:08 7 hours ago	#2362529786 Merge branch 'feature/add-new-features...' development -> 8d4cb02b		✓ ✗	↺ ↓ ⌵
Failed 00:01:10 2 months ago	#2210689671 [UPDATE] ACCESS_LOOGING_EXCLUDE... development -> 126360cd		✓ ✗	↺ ↓ ⌵
Failed 00:01:43 3 months ago	#2184584261 Merge branch 'development' of https://gi... development -> 7d990f70		✓ ✗	↺ ↓ ⌵
Failed 00:01:14 4 months ago	#2129252107 Merge branch 'feature/video-attachment'... development -> 9de09bfb		✓ ✗	↺ ↓ ⌵
Failed 00:07:54 4 months ago	#2113707048 Merge branch 'feature/add-validate-toast'... development -> e6276d36		✓ ✗	↺ ↓ ⌵
Passed 00:07:51 4 months ago	#2113190143 [CI] add sonarqube development -> 54828219		✓ ✓	↓ ⌵
Passed 00:00:39 4 months ago	#2113187432 [ADD] README docs development -> afb14b8c		✓	↓ ⌵

ตัวอย่าง CI Pipeline บน Gitlab

ขั้นตอนของ CI (CI Pipeline)



CD (Continuous Delivery & Deployment) คืออะไร

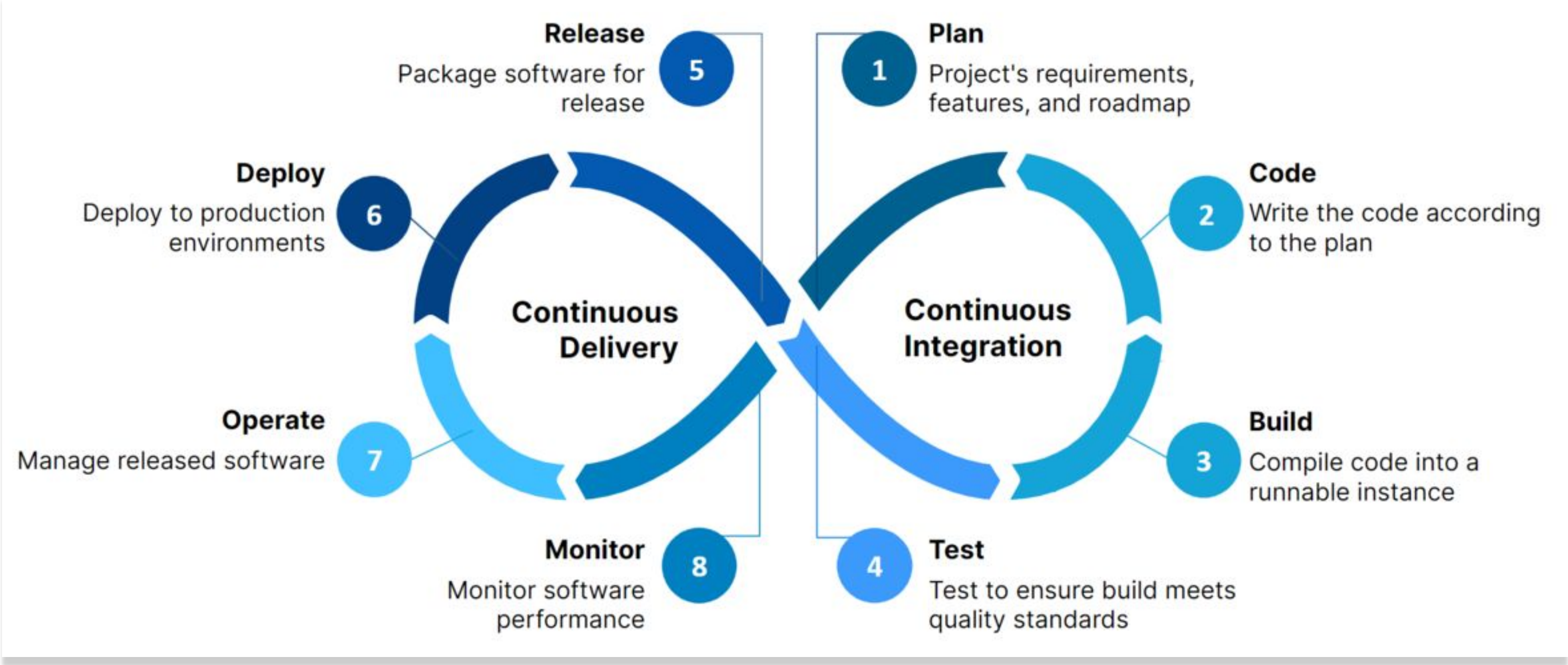
Continuous Delivery

ระบบเตรียมซอฟต์แวร์ให้พร้อม Deploy ตลอดเวลา เช่น สร้าง Docker Image รอไว้ แต่การขึ้น Production จริงต้องมี **คนกดอนุมัติ (Manual Gate)**

Continuous Deployment

ระบบนำโค้ดขึ้น Production อัตโนมัติทันทีที่ผ่านการทดสอบ (Fully Automated)

CI/CD



ประโยชน์ของ CI/CD



Speed

ส่งมอบฟีเจอร์ใหม่ได้รวดเร็ว ลด
Time-to-Market



Quality

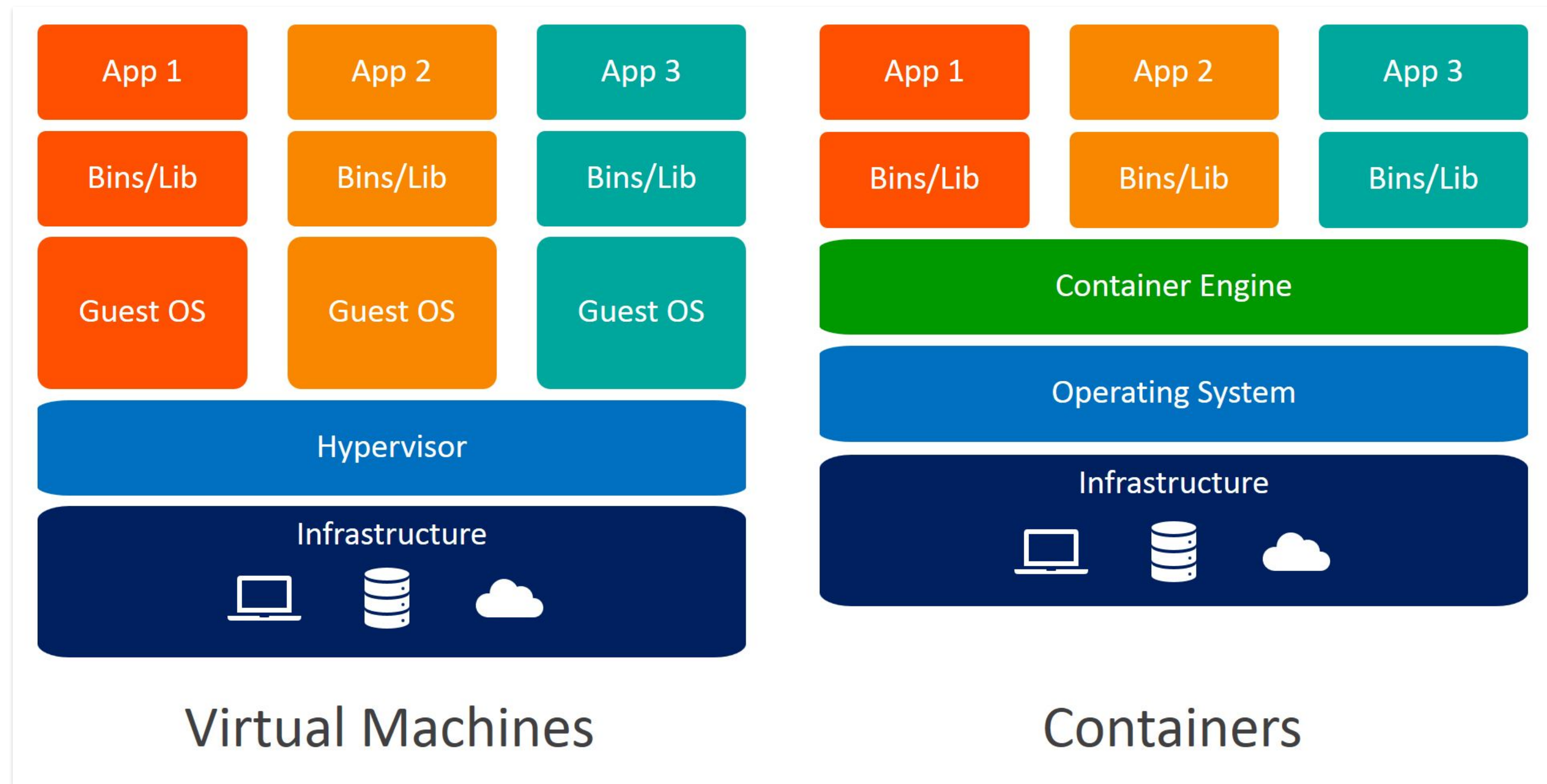
คุณภาพโค้ดดีขึ้น ลดบั๊กที่จะหลุด
ไปถึงผู้ใช้



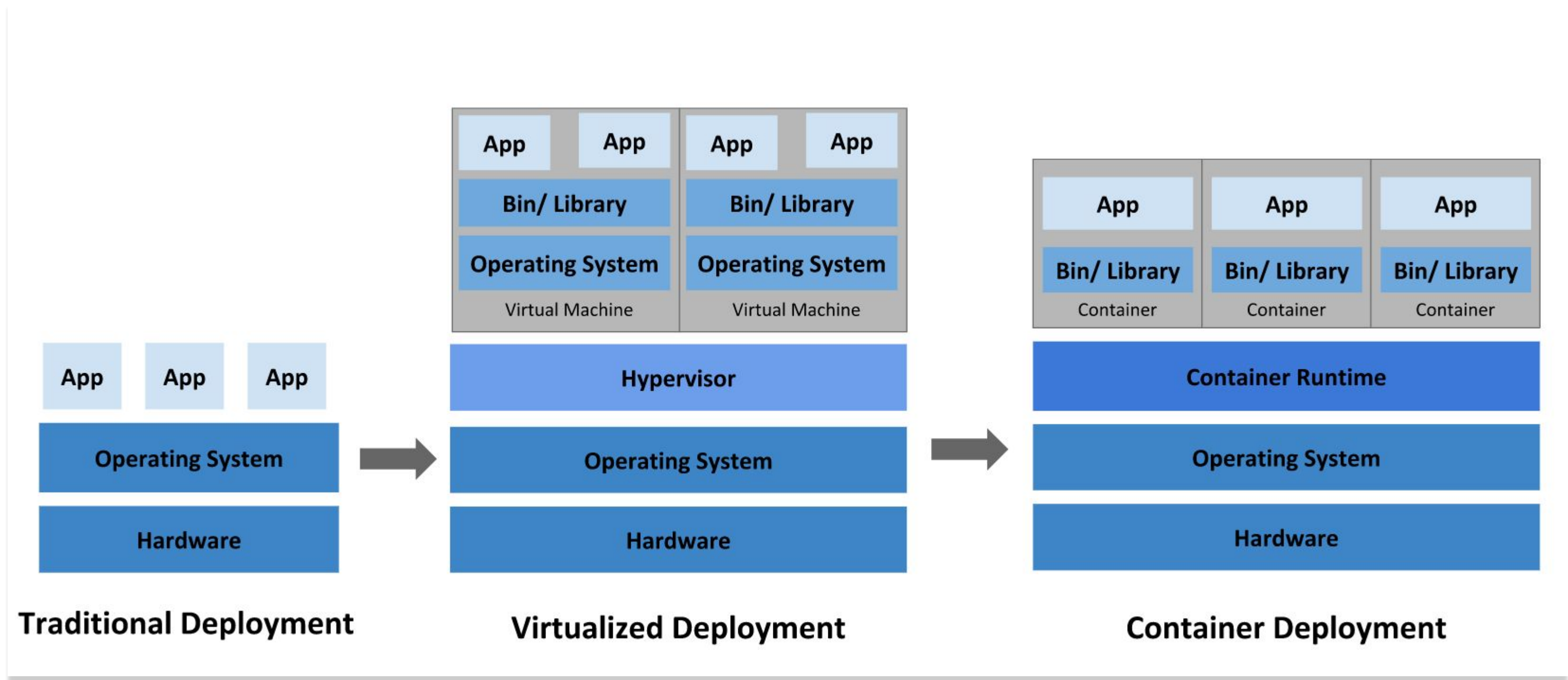
Reliability

Deploy ได้บ่อยและมั่นคงขึ้น คุ้ม
ค่านระบบได้ไว

จาก Virtual Machines สู่ Containers



จาก Virtual Machines สู่ Containers



จาก Virtual Machines สู่ Containers

Virtual Machines (VM)

- ✗ หนัก ใช้ Ram/Disk สูง
- ✗ บูตช้า (s)
- ✗ มี OS ซ้อน OS (Hypervisor)

Containers

- ✓ เบา ขนาดเล็ก เร็ว รันได้เยอะ
- ✓ บูตในเสี้ยววินาที (ms)
- ✓ แชร์ OS Kernel ร่วมกัน
- ✓ Environment เหมือนกัน (Consistency)

Docker: มาตรฐานของ Container



Dockerfile

Script ที่ระบุขั้นตอนสร้าง Image



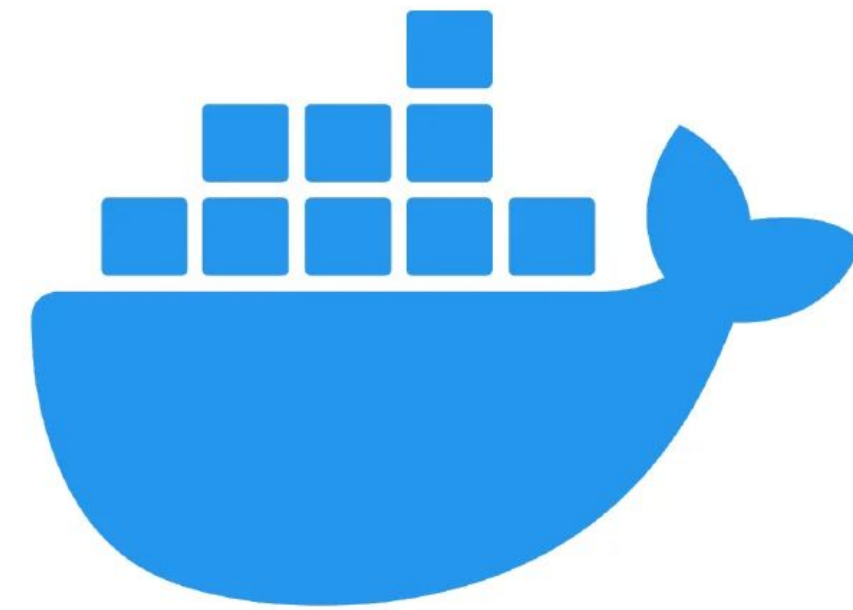
Image

ผลลัพธ์จากการ Build ที่เป็นไฟล์ Read-only พร้อมนำไปใช้งาน



Container

Image ที่ถูกนำมารันจริง (Instance) บน OS ใดก็ได้ที่มี Docker



"Build Once, Run Anywhere"

สร้างครั้งเดียว รันได้ทุกที่ ไม่ต้องกังวลเรื่อง Environment

ทำไมต้องมี Container Orchestration?

1000

Scalability

Managing thousands of containers manually is impossible.
Orchestrators handle scaling automatically.

99.9

High Availability

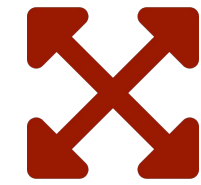
Self-healing: If a container fails, the orchestrator restarts it
instantly to maintain uptime.

ทำไมต้องมี Container Orchestration?

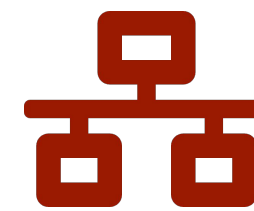
- จะอัปเดตแอปอย่างไรไม่ให้ระบบล่ม
- จะรัน Container ที่เครื่องไหนดี? (Scheduling)
- Container ตัวไหนตายจะชุบชีวิตยังไง? (Self-healing)
- จะกระจาย Traffic คนเข้าใช้งานอย่างไร? (Load Balancing)
- จะขยายระบบ (Scale) ตอนคนเข้าเยอะๆ อย่างไร?



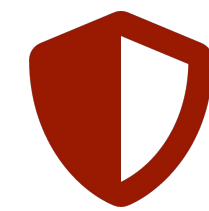
Health Management



Auto Scaling



Load Balancing



Service Discovery

ความสามารถหลักของ Kubernetes (K8s)



Self - Healing

ถ้า Container พัง K8s จะสั่งรันตัวใหม่ขึ้นมาแทน
ที่โดยอัตโนมัติทันที



Auto - Scaling

เพิ่ม/ลดจำนวน Container ตามภาระงาน (Load)
จริงได้แบบ Real-time



Rolling Updates

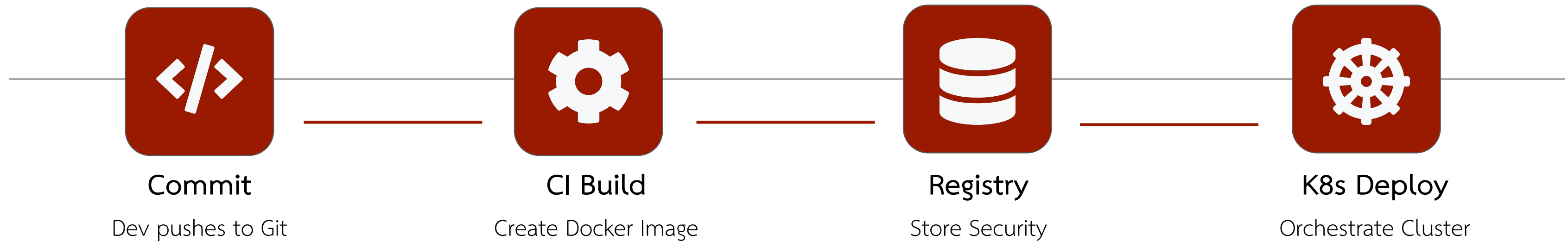
อัปเดตเวอร์ชันใหม่แบบไม่มี Downtime และถอย
กลับได้ทันทีถ้ามีปัญหา



ระบบ Orchestration ที่ถูกพัฒนาโดย

Google

เมื่อ CI/CD เจอกับ Container Orchestration



เครื่องมือที่นิยมใช้ในปัจจุบัน

Category	Popular Tools
CI/CD Platforms	GitHub Actions, GitLab CI, Jenkins
Containerization	Docker
Orchestration	Kubernetes (K8s), Amazon ECS
Infrastructure as Code	Pulumi, Helm

WORKSHOP 01

การสร้าง Docker Container และ K3s Pods

Prerequisites

- Docker : <https://www.docker.com/products/docker-desktop/>
- Lens : <https://lenshq.io/download>
- Helm: <http://helm.sh/docs/intro/install/>
- Python 3.9+ : <https://www.python.org/downloads/>

Ansible

- ansible : https://docs.ansible.com/projects/ansible/latest/installation_guide/intro_installation.html
- python3 -m pip install --user ansible
- ansible-galaxy collection install kubernetes.core

SSH

https://drive.google.com/drive/folders/1fd0txlR77GrO9c5p2ikV9R02stkKJtzv?usp=drive_link

Git Repository

<https://github.com/wasit7/Living-Lab> (2_auto_ops)

```
git clone --depth 1 --filter=blob:none --sparse https://github.com/wasit7/Living-Lab.git && cd Living-Lab && git  
sparse-checkout set 2_auto_ops
```


docker-compose.yml

- version — ระบุว่าเราจะใช้ Compose file เวอร์ชันไหน
- services — ระบุ container ที่จะต้องใช้
- image — เรียกใช้ Image จาก Docker Hub Registry
- ports — ทำ port mapping ระหว่าง host กับ container
- volumes — การสร้าง volumes มี 2 แบบ ซึ่งถ้าสร้างอยู่ภายในชื่อ service แต่ละตัวก็คือการเชื่อม volume แต่ถ้าอยู่ในระดับเดียวกับ services: จะเป็นการสร้าง volume
- build — การบอกว่าให้ใช้ image ที่สร้างจาก Dockerfile
- links — เป็นการผูก service เข้าด้วยกัน ทำให้ service สามารถเรียกใช้งาน service ที่ link ได้
- network — เป็นการใช้เพื่อสร้างเส้นทางสื่อสารกันระหว่าง container
- context — path ของ dockerfile เพื่อที่จะใช้ในการสร้าง container
- environment — การกำหนดค่าตัวแปรสภาพแวดล้อมลงไปตรงๆ ในไฟล์ เช่น กำหนดรหัสผ่าน
- env_file — ดึงค่าตัวแปรมาจากไฟล์ภายนอก (เช่น ไฟล์ .env) ช่วยให้ไฟล์ compose สะอาดขึ้น และซ่อนความลับได้ดีกว่าการพิมพ์ตรง ๆ
- depends_on — สั่งให้ service นั้นเริ่มทำงานหลังจาก service ที่ depends_on อยู่เริ่มต้นทำงานเสร็จแล้ว

Docker & Docker Compose CLI ที่ต้องรู้

`docker ps`: ดู Container ที่กำลังรันอยู่

`docker-compose up -d`: สั่งรันทุก Service ในไฟล์แบบ Background (ถ้ายังไม่มี Image จะ Build/Pull ให้เอง)

`docker-compose down`: สั่งหยุดและ ลบ Container รวมถึง Network ที่สร้างขึ้น (ข้อมูลใน Volume ไม่หาย)

`docker-compose ps`: ดูสถานะของทุก Service ในโปรเจกต์

`docker-compose logs -f [service_name]`: ดู Log รวม หรือระบุชื่อ Service เฉพาะตัว

`docker-compose restart`: เริ่มการทำงานของ Service ใหม่ทั้งหมด

`docker-compose build --no-cache`: บังคับ Build Image ใหม่ทั้งหมดโดยไม่ใช้ Cache เดิม

Ansible

```
ansible-playbook -i inventory/hosts.yml site.yml
```

Value.yml

backend:

image: docker-workshop-backend:local

frontend:

image: docker-workshop-frontend:local

database:

user: workshop_user

password: workshop_password

name: workshop_db

แก้ปัญหา SSL Certificate

```
sudo nano /etc/systemd/system/k3s.service
```

```
--tls-san 35.187.252.158
```

```
sudo systemctl daemon-reload
```

```
sudo systemctl restart k3s
```


docker image

```
docker save docker-workshop-frontend:local | sudo /usr/local/bin/k3s ctr --address /run/containerd/containerd.sock  
images import -
```

```
docker save docker-workshop-backend:local | sudo /usr/local/bin/k3s ctr --address /run/containerd/containerd.sock  
images import -
```

```
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl delete pod --all -n workshop-livinglab
```


Deploy App

```
kubectl create namespace workshop-livinglab
```

```
helm upgrade --install docker-workshop ./workshop-chart --namespace workshop-livinglab
```

WORKSHOP 02

การตั้งค่า CI/CD Pipeline บน GitLab

ติดตั้ง GitLab Runner ใน VM

```
sudo curl -L --output /usr/local/bin/gitlab-runner
```

```
https://gitlab-runner-downloads.s3.amazonaws.com/latest/binaries/gitlab-runner-linux-amd64
```

```
sudo chmod +x /usr/local/bin/gitlab-runner
```

```
sudo useradd --comment 'GitLab Runner' --create-home gitlab-runner --shell /bin/bash
```

```
sudo gitlab-runner install --user=gitlab-runner --working-directory=/home/gitlab-runner
```

```
sudo gitlab-runner start
```

Register GitLab Runner

```
gitlab-runner register --url https://gitlab.com --token xxxxxxxxx
```

```
gitlab-runner run
```

```
sudo usermod -aG docker gitlab-runner
```

```
sudo chmod 644 /etc/rancher/k3s/k3s.yaml
```

```
sudo systemctl restart gitlab-runner
```

แก้ปัญหา GitLab Runner

```
sudo rm /home/gitlab-runner/.bash_logout
```

```
sudo mv /home/gitlab-runner/.bash_logout /home/gitlab-runner/.bash_logout.bak 2>/dev/null
```

```
sudo mv /home/gitlab-runner/.bashrc /home/gitlab-runner/.bashrc.bak 2>/dev/null
```

```
sudo mv /home/gitlab-runner/.profile /home/gitlab-runner/.profile.bak 2>/dev/null
```


.gitlab-ci.yml

deploy_job:

stage: deploy

tags:

- workshop-runner

script:

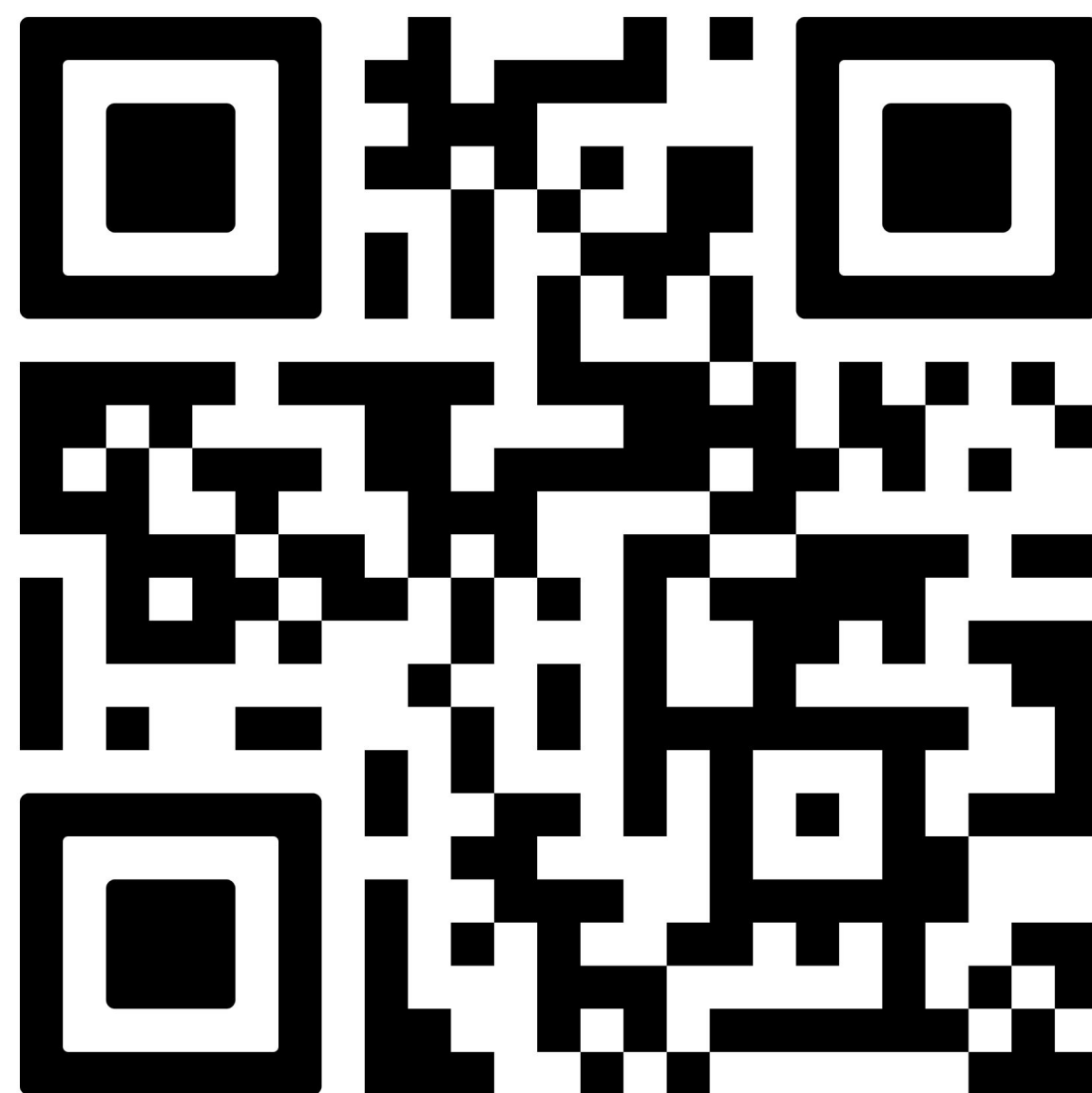
- cd docker-workshop
- docker compose build
- export KUBECONFIG=/etc/rancher/k3s/k3s.yaml
- helm upgrade --install docker-workshop ./workshop-chart --namespace workshop-livinglab
- kubectl rollout restart deployment frontend -n workshop-livinglab
- kubectl rollout restart deployment backend -n workshop-livinglab


```
73 10:57:14 #20 exporting attestation manifest sha256:d8602cd6bd63eb0000b06e8151aa7433a76d0f69f5cdcafa549e8a7537478586 0.1s done
74 10:57:14 #20 exporting manifest list sha256:c980ce6e5839a62d4651cbdb12a6f7308bdd34814b52fa1c86707930ed61380c 0.0s done
75 10:57:14 #20 naming to docker.io/library/workshop-backend:local done
76 10:57:14 #20 unpacking to docker.io/library/workshop-backend:local
77 10:57:14 #20 unpacking to docker.io/library/workshop-backend:local 0.0s done
78 10:57:14 #20 DONE 0.2s
79 10:57:14 #21 [frontend] exporting to image
80 10:57:14 #21 exporting layers 0.0s done
81 10:57:14 #21 exporting manifest sha256:c371969840336759a8c5de2b9a4677d6823867f475ac428eece5281003714725 done
82 10:57:14 #21 exporting config sha256:c7a512765cc09b460512f48a39c9320666284bc7c039b4ddf7773c76697f7e27 done
83 10:57:14 #21 exporting attestation manifest sha256:1985ac80cee2268535395b15e4005ae60018c9b0baf5e61b58d9f1ba29e82913 0.1s done
84 10:57:14 #21 exporting manifest list sha256:85785196f2ff3c811ca4b8ff7ded31a0714aebc3566813ceb4785f7f9c912329 0.0s done
85 10:57:14 #21 naming to docker.io/library/workshop-frontend:local done
86 10:57:14 #21 unpacking to docker.io/library/workshop-frontend:local 0.0s done
87 10:57:14 #21 DONE 0.2s
88 10:57:14 #22 [frontend] resolving provenance for metadata file
89 10:57:14 #22 DONE 0.0s
90 10:57:14 #23 [backend] resolving provenance for metadata file
91 10:57:14 #23 DONE 0.0s
92 10:57:14 Image workshop-backend:local Built
93 10:57:14 Image workshop-frontend:local Built
94 10:57:14 $ export KUBECONFIG=/etc/rancher/k3s/k3s.yaml
95 10:57:14 $ helm upgrade --install docker-workshop ./workshop-chart --namespace workshop-livinglab
96 10:57:15 Release "docker-workshop" has been upgraded. Happy Helming!
97 10:57:15 NAME: docker-workshop
98 10:57:15 LAST DEPLOYED: Fri Mar 6 10:57:15 2026
99 10:57:15 NAMESPACE: workshop-livinglab
100 10:57:15 STATUS: deployed
101 10:57:15 REVISION: 16
102 10:57:15 TEST SUITE: None
103 10:57:15 $ kubectl rollout restart deployment frontend -n workshop-livinglab
104 10:57:15 deployment.apps/frontend restarted
105 10:57:15 $ kubectl rollout restart deployment backend -n workshop-livinglab
106 10:57:16 deployment.apps/backend restarted
107 10:57:16 Cleaning up project directory and file based variables
108 10:57:16 Job succeeded
```

00:00



แบบประเมินความพึงพอใจ



<https://forms.gle/GipMD5drZHA7LU4n6>